Text-to-SQL Architecture with LLMs

Overview

Natural language to SQL query generation system using LLMs, enabling non-technical users to query databases conversationally.

Architecture Components

1. System Flow

```
User Question (Natural Language)
  → Schema Understanding (RAG)
  → LLM SQL Generation (Bedrock/OpenAI)
  → SQL Validation & Safety Check
  → Query Execution (Read-only)
  → Result Formatting
  → Natural Language Response
  → Visualization (optional)
```

2. Core Components

Schema Context Management

```python
from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import create_sql_agent
from langchain_community.llms import Bedrock
from langchain.prompts import PromptTemplate

# Connect to database
db = SQLDatabase.from_uri(
    "postgresql://user:pass@host:5432/dbname",
    include_tables=['customers', 'orders', 'products'],
    sample_rows_in_table_info=3
)

# Get schema information
schema_info = db.get_table_info()
print(schema_info)

# Output:
# Table: customers
# Columns: id (INTEGER), name (VARCHAR), email (VARCHAR), created_at (TIMESTAMP)
# Sample rows:
```

```
# (1, 'John Doe', 'john@example.com', '2024-01-01 00:00:00')
# ...
```

Schema Storage with Vector DB

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import BedrockEmbeddings
from langchain_community.vectorstores import Pinecone

# Store table schemas with descriptions
table_schemas = [
    {
        "table": "customers",
        "description": "Customer information including contact details and
registration date",
        "columns": {
            "id": "Primary key, auto-increment",
            "name": "Customer full name",
            "email": "Customer email address",
            "created_at": "Account creation timestamp"
        },
        "sample_queries": [
            "SELECT * FROM customers WHERE email LIKE '%@gmail.com'",
            "SELECT COUNT(*) FROM customers WHERE created_at > '2024-01-01'"
        ]
    },
    {
        "table": "orders",
        "description": "Customer orders with status and amounts",
        "columns": {
            "id": "Order ID",
            "customer_id": "Foreign key to customers.id",
            "amount": "Order total amount in USD",
            "status": "Order status: pending, completed, cancelled",
            "order_date": "Date order was placed"
        }
    }
]

# Create embeddings and store in vector DB
embeddings = BedrockEmbeddings(model_id="amazon.titan-embed-text-v1")

# Convert schemas to text
schema_texts = []
for schema in table_schemas:
    text = f"""
    Table: {schema['table']}
    Description: {schema['description']}
    Columns: {', '.join([f"{k}: {v}" for k, v in schema['columns'].items()])}
    """
    schema_texts.append(text)
```

```
vectorstore = Pinecone.from_texts(
    texts=schema_texts,
    embedding=embeddings,
    index_name="sql-schema"
)
```

3. SQL Generation Approaches

Approach 1: Direct LLM (Simple)

```
from langchain_community.llms import Bedrock
from langchain.prompts import PromptTemplate

llm = Bedrock(
    model_id="anthropic.claude-3-sonnet-20240229-v1:0",
    model_kwargs={"max_tokens": 1000}
)

prompt_template = """Given the following database schema:

{schema}

Write a SQL query to answer this question: {question}

Requirements:
- Use only SELECT statements (no INSERT, UPDATE, DELETE)
- Include appropriate WHERE clauses for filtering
- Use JOINs when multiple tables are needed
- Return only the SQL query, no explanations

SQL Query:"""

prompt = PromptTemplate(
    template=prompt_template,
    input_variables=["schema", "question"]
)

chain = prompt | llm

sql_query = chain.invoke({
    "schema": schema_info,
    "question": "How many customers signed up last month?"
})
```

Approach 2: LangChain SQL Agent (Recommended)

```
from langchain_community.agent_toolkits import create_sql_agent
from langchain.agents.agent_types import AgentType

agent_executor = create_sql_agent(
    llm=llm,
    db=db,
    agent_type=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True,
    handle_parsing_errors=True
)

response = agent_executor.invoke({
    "input": "What are the top 5 customers by total order value?"
})

print(response['output'])
```

Approach 3: Few-Shot Learning

```
few_shot_examples = [
    {
        "question": "How many customers do we have?",
        "sql": "SELECT COUNT(*) FROM customers;"
    },
    {
        "question": "What were the total sales last month?",
        "sql": """SELECT SUM(amount)
                  FROM orders
                  WHERE order_date >= DATE_TRUNC('month', CURRENT_DATE - INTERVAL
'1 month')
                  AND order_date < DATE_TRUNC('month', CURRENT_DATE);"""
    },
    {
        "question": "Who are our top 10 customers by order count?",
        "sql": """SELECT c.name, COUNT(o.id) as order_count
                  FROM customers c
                  JOIN orders o ON c.id = o.customer_id
                  GROUP BY c.id, c.name
                  ORDER BY order_count DESC
                  LIMIT 10;"""
    }
]

from langchain.prompts import FewShotPromptTemplate, PromptTemplate

example_prompt = PromptTemplate(
    input_variables=["question", "sql"],
    template="Question: {question}\nSQL: {sql}"
)
```

```
few_shot_prompt = FewShotPromptTemplate(
    examples=few_shot_examples,
    example_prompt=example_prompt,
    prefix="You are a SQL expert. Given a question, write a SQL query.\n\n",
    suffix="Question: {question}\nSQL:",
    input_variables=["question"]
)
```

Approach 4: RAG + SQL Generation

```
from langchain.chains import RetrievalQA

# Retrieve relevant schema information
def generate_sql_with_rag(question: str):
    # 1. Retrieve relevant tables/columns
    relevant_schemas = vectorstore.similarity_search(question, k=3)

    # 2. Get sample queries
    context = "\n".join([doc.page_content for doc in relevant_schemas])

    # 3. Generate SQL
    prompt = f"""Based on this schema information:

    {context}

    Database Schema:
    {schema_info}

    Generate a SQL query to answer: {question}

    SQL Query:"""

    sql_query = llm.invoke(prompt)

    return sql_query.strip()
```

4. SQL Validation & Safety

```
import sqlparse
from sqlparse.sql import Token
import re

class SQLValidator:

    FORBIDDEN_KEYWORDS = [
        'DROP', 'DELETE', 'INSERT', 'UPDATE', 'ALTER',
        'CREATE', 'TRUNCATE', 'GRANT', 'REVOKE'
    ]
```

```
    @staticmethod
    def is_safe_query(sql: str) -> tuple[bool, str]:
        """Validate SQL query for safety."""

        # Parse SQL
        parsed = sqlparse.parse(sql)

        if not parsed:
            return False, "Invalid SQL"

        statement = parsed[0]

        # Check if SELECT only
        if statement.get_type() != 'SELECT':
            return False, "Only SELECT queries are allowed"

        # Check for forbidden keywords
        sql_upper = sql.upper()
        for keyword in SQLValidator.FORBIDDEN_KEYWORDS:
            if keyword in sql_upper:
                return False, f"Forbidden keyword: {keyword}"

        # Check for multiple statements
        if len(parsed) > 1:
            return False, "Multiple statements not allowed"

        # Check for comments (potential SQL injection)
        if '--' in sql or '/*' in sql:
            return False, "Comments not allowed"

        return True, "Valid"

    @staticmethod
    def add_row_limit(sql: str, limit: int = 1000) -> str:
        """Add LIMIT clause if not present."""
        sql_upper = sql.upper()

        if 'LIMIT' not in sql_upper:
            sql = sql.rstrip(';')
            sql = f"{sql} LIMIT {limit};"

        return sql

    @staticmethod
    def estimate_cost(sql: str, db) -> dict:
        """Estimate query cost using EXPLAIN."""
        try:
            explain_query = f"EXPLAIN (FORMAT JSON) {sql}"
            result = db.run(explain_query)
            return json.loads(result)[0]['Plan']
        except Exception as e:
            return {"error": str(e)}

# Usage
```

```
validator = SQLValidator()

sql = "SELECT * FROM customers WHERE name = 'John'"
is_safe, message = validator.is_safe_query(sql)

if is_safe:
    sql = validator.add_row_limit(sql)
    cost = validator.estimate_cost(sql, db)

    if cost.get('Total Cost', 0) < 10000:  # Threshold
        result = db.run(sql)
    else:
        print("Query too expensive")
else:
    print(f"Unsafe query: {message}")
```

5. Query Execution & Result Handling

```
import pandas as pd
from typing import Any, Dict, List

class QueryExecutor:

    def __init__(self, db):
        self.db = db

    def execute_with_timeout(self, sql: str, timeout: int = 30) -> pd.DataFrame:
        """Execute query with timeout."""
        import signal

        def timeout_handler(signum, frame):
            raise TimeoutError("Query execution timeout")

        # Set timeout
        signal.signal(signal.SIGALRM, timeout_handler)
        signal.alarm(timeout)

        try:
            result = self.db.run(sql)
            signal.alarm(0)  # Cancel alarm

            # Convert to DataFrame
            df = pd.read_sql(sql, self.db._engine)
            return df

        except TimeoutError:
            raise
        except Exception as e:
            raise Exception(f"Query execution failed: {str(e)}")

    def format_results(self, df: pd.DataFrame) -> Dict[str, Any]:
```

```
        """Format results for display."""

        return {
            "row_count": len(df),
            "columns": list(df.columns),
            "data": df.to_dict('records'),
            "summary": {
                "numeric_columns": df.select_dtypes(include=
['number']).describe().to_dict(),
                "categorical_columns": {
                    col: df[col].value_counts().head(10).to_dict()
                    for col in df.select_dtypes(include=['object']).columns
                }
            }
        }

    def execute_and_explain(self, sql: str) -> Dict[str, Any]:
        """Execute query and provide explanation."""

        # Execute
        df = self.execute_with_timeout(sql)

        # Format results
        results = self.format_results(df)

        # Generate natural language explanation
        explanation_prompt = f"""
        SQL Query: {sql}

        Results: {len(df)} rows returned
        Columns: {', '.join(df.columns)}

        Provide a brief, natural language summary of these results.
        """

        explanation = llm.invoke(explanation_prompt)

        return {
            "sql": sql,
            "results": results,
            "explanation": explanation
        }

# Usage
executor = QueryExecutor(db)
result = executor.execute_and_explain(sql_query)
```

6. AWS Architecture for Text-to-SQL

```
┌─────────────────────────────────────────────────────────────┐
│  User Interface                                             │
```

```
│  - Web App (React/Vue)                                          │
│  - Slack Bot                                                    │
│  - API                                                          │
└─────────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────────┐
│  API Gateway + Lambda (Query Handler)                           │
│  - Input validation                                             │
│  - User authentication                                          │
│  - Rate limiting                                                │
└─────────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────────┐
│  Schema Context Service                                         │
│  - Pinecone/OpenSearch (schema embeddings)                      │
│  - DynamoDB (metadata cache)                                    │
│  - ElastiCache Redis (query cache)                              │
└─────────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────────┐
│  SQL Generation Service (Lambda/ECS)                            │
│  - LangChain SQL Agent                                          │
│  - Bedrock Claude 3 Sonnet                                      │
│  - Few-shot examples                                            │
└─────────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────────┐
│  Validation & Safety Layer                                      │
│  - SQL parsing (sqlparse)                                       │
│  - Query cost estimation                                        │
│  - Row limit enforcement                                        │
└─────────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────────┐
│  Database Access (Read Replicas)                                │
│  - Aurora Read Replica                                          │
│  - Redshift                                                     │
│  - Athena                                                       │
│  - RDS Proxy (connection pooling)                               │
└─────────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────────┐
│  Result Processing                                              │
│  - Data formatting                                              │
│  - Visualization (QuickSight/Charts)                            │
│  - Natural language summary                                     │
└─────────────────────────────────────────────────────────────────┘
```

7. Advanced Features

Multi-Turn Conversations

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain

memory = ConversationBufferMemory()

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

# Turn 1
response1 = conversation.predict(
    input="Show me customers who signed up last month"
)
# Generates: SELECT * FROM customers WHERE created_at >= ...

# Turn 2 (follow-up)
response2 = conversation.predict(
    input="Now filter to only those from California"
)
# Modifies previous query: ... AND state = 'CA'

# Turn 3 (clarification)
response3 = conversation.predict(
    input="Order by signup date"
)
# Adds: ORDER BY created_at DESC
```

Query Optimization Suggestions

```
def suggest_optimizations(sql: str, db) -> List[str]:
    """Suggest query optimizations."""
    suggestions = []

    # Get execution plan
    explain = db.run(f"EXPLAIN (ANALYZE, BUFFERS) {sql}")

    # Parse plan
    if 'Seq Scan' in explain:
        suggestions.append("Consider adding an index for better performance")

    if 'Sort' in explain and 'LIMIT' in sql.upper():
        suggestions.append("Consider using an index to avoid sorting")

    # Check for missing JOINs
    if 'WHERE' in sql.upper() and '=' in sql:
        suggestions.append("Review if JOIN would be more appropriate than WHERE")
```

```
    return suggestions
```

Intelligent Error Recovery

```
def execute_with_recovery(question: str, max_attempts: int = 3) -> Dict:
    """Execute query with automatic error recovery."""

    for attempt in range(max_attempts):
        try:
            # Generate SQL
            sql = generate_sql(question)

            # Validate
            is_safe, message = validator.is_safe_query(sql)
            if not is_safe:
                raise ValueError(f"Unsafe query: {message}")

            # Execute
            result = executor.execute_with_timeout(sql)
            return {"success": True, "result": result, "sql": sql}

        except Exception as e:
            if attempt < max_attempts - 1:
                # Ask LLM to fix the error
                fix_prompt = f"""
                The following SQL query failed with error: {str(e)}

                SQL: {sql}

                Please provide a corrected version.
                """
                sql = llm.invoke(fix_prompt)
            else:
                return {
                    "success": False,
                    "error": str(e),
                    "attempts": attempt + 1
                }
```

8. Monitoring with LangSmith

```
import os
from langsmith import Client
from langsmith.run_helpers import traceable

os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_PROJECT"] = "text-to-sql-prod"
```

```
client = Client()

@traceable(name="text-to-sql-query")
def process_query(question: str, user_id: str):
    """Process text-to-SQL query with full tracing."""

    with client.trace("retrieve-schema", metadata={"user_id": user_id}):
        schema_context = get_schema_context(question)

    with client.trace("generate-sql", metadata={"user_id": user_id}):
        sql = generate_sql(question, schema_context)

    with client.trace("validate", metadata={"sql": sql}):
        is_safe, message = validator.is_safe_query(sql)

    if is_safe:
        with client.trace("execute", metadata={"sql": sql}):
            result = executor.execute_with_timeout(sql)

        return result
    else:
        raise ValueError(f"Unsafe query: {message}")
```

9. Complete Implementation Example

```
# app.py - Full Text-to-SQL Application

import boto3
from langchain_community.utilities import SQLDatabase
from langchain_community.llms import Bedrock
from langchain_community.agent_toolkits import create_sql_agent
from langchain.prompts import PromptTemplate
import sqlparse

class TextToSQLService:

    def __init__(self, db_uri: str, model_id: str = "anthropic.claude-3-sonnet-
20240229-v1:0"):
        self.db = SQLDatabase.from_uri(db_uri)
        self.llm = Bedrock(model_id=model_id)
        self.agent = create_sql_agent(llm=self.llm, db=self.db, verbose=True)
        self.validator = SQLValidator()

    def query(self, question: str, user_id: str) -> dict:
        """Main query method."""

        try:
            # Check cache
            cached_result = self.check_cache(question)
            if cached_result:
                return cached_result
```

```
            # Generate SQL
            response = self.agent.invoke({"input": question})
            sql = self.extract_sql(response)

            # Validate
            is_safe, message = self.validator.is_safe_query(sql)
            if not is_safe:
                return {
                    "success": False,
                    "error": f"Unsafe query: {message}"
                }

            # Add safety limits
            sql = self.validator.add_row_limit(sql)

            # Execute
            df = pd.read_sql(sql, self.db._engine)

            # Format and explain
            result = {
                "success": True,
                "sql": sql,
                "row_count": len(df),
                "data": df.to_dict('records'),
                "explanation": response['output']
            }

            # Cache result
            self.cache_result(question, result)

            # Log for monitoring
            self.log_query(user_id, question, sql, success=True)

            return result

        except Exception as e:
            self.log_query(user_id, question, None, success=False, error=str(e))
            return {
                "success": False,
                "error": str(e)
            }

    def extract_sql(self, agent_response: dict) -> str:
        """Extract SQL from agent response."""
        output = agent_response.get('output', '')

        # Look for SQL in code blocks
        if '```sql' in output:
            sql = output.split('```sql')[1].split('```')[0].strip()
            return sql
        elif 'SELECT' in output.upper():
            # Extract first SELECT statement
            parsed = sqlparse.parse(output)
```

```
            return str(parsed[0])
        else:
            raise ValueError("No SQL found in response")

    def check_cache(self, question: str) -> dict:
        """Check Redis cache for similar queries."""
        # Implement semantic caching
        pass

    def cache_result(self, question: str, result: dict):
        """Cache query result."""
        pass

    def log_query(self, user_id, question, sql, success, error=None):
        """Log to CloudWatch/DynamoDB."""
        dynamodb = boto3.resource('dynamodb')
        table = dynamodb.Table('text-to-sql-logs')

        table.put_item(Item={
            'query_id': str(uuid.uuid4()),
            'user_id': user_id,
            'question': question,
            'sql': sql or '',
            'success': success,
            'error': error or '',
            'timestamp': datetime.now().isoformat()
        })

# Lambda handler
def lambda_handler(event, context):
    service = TextToSQLService(
        db_uri=os.environ['DATABASE_URL']
    )

    body = json.loads(event['body'])
    question = body['question']
    user_id = event['requestContext']['authorizer']['claims']['sub']

    result = service.query(question, user_id)

    return {
        'statusCode': 200 if result['success'] else 400,
        'body': json.dumps(result)
    }
```

10. Cost Optimization

```
# Intelligent caching with embeddings
from langchain.cache import RedisSemanticCache

cache = RedisSemanticCache(
```

```
    redis_url="redis://your-elasticache",
    embedding=embeddings,
    score_threshold=0.95
)

# Query deduplication
from hashlib import md5

def get_query_fingerprint(question: str) -> str:
    """Generate fingerprint for similar queries."""
    # Normalize question
    normalized = question.lower().strip()
    normalized = re.sub(r'\s+', ' ', normalized)

    return md5(normalized.encode()).hexdigest()

# Result pagination
def paginate_results(df: pd.DataFrame, page: int = 1, page_size: int = 100):
    """Paginate large result sets."""
    start = (page - 1) * page_size
    end = start + page_size

    return {
        "data": df.iloc[start:end].to_dict('records'),
        "page": page,
        "page_size": page_size,
        "total_rows": len(df),
        "total_pages": (len(df) + page_size - 1) // page_size
    }
```

Best Practices

1. **Always use read-only database connections**
2. **Implement query timeouts (30s recommended)**
3. **Add row limits to prevent large result sets**
4. **Cache frequent queries**
5. **Monitor query costs and patterns**
6. **Provide example questions to users**
7. **Maintain up-to-date schema documentation**
8. **Use LangSmith for debugging and optimization**
9. **Implement user-level query quotas**
10. **Regularly review and update few-shot examples**

Common Challenges & Solutions

Challenge	Solution
Ambiguous questions	Ask clarifying questions before generating SQL

Challenge	Solution
Complex joins	Store common join patterns in few-shot examples
Performance issues	Use EXPLAIN to suggest indexes
Incorrect results	Add validation step comparing results to expectations
Database differences	Maintain dialect-specific prompts

Deployment Checklist

- [ ] Database read replica configured
- [ ] Schema documentation complete
- [ ] Few-shot examples prepared
- [ ] SQL validation implemented
- [ ] Query timeout configured
- [ ] Result caching enabled
- [ ] Monitoring dashboard created
- [ ] User permissions configured
- [ ] Rate limiting enabled
- [ ] Error handling tested